

Name: _____

Date: _____

Mr. Rawson • Y8 MYP Design



The Games Menu

You are going to pick one of these, and build it.

Everything on this menu is a real micro:bit project. People have built all of them. You can too.

The games are in three groups. **Harder is not better.** A simple game that works, that somebody actually wants to play, and that you can explain, beats an ambitious one that never quite runs.

Three questions before you choose

1. **Would you play it?** If the honest answer is no, pick another one.
2. **Who else would play it?** A friend? A little brother? The whole class at once? You will be asked this again later, so have an answer.
3. **Can you explain how it works?** Not the code — the *idea*. If you cannot say it in one sentence, it is too big.

The marks on the cards:

★ There is a full step-by-step tutorial that walks you through every block. Everything else hands you the finished code to read, which is harder — and more interesting.

❓ This one has a real **question** buried in it that nobody has answered for you. Building it is only half the job.

(radio) You need two or more micro:bits and a partner.

The best games on this menu are not the hardest ones. They are the ones where the micro:bit is a small part of a game that real people play, standing up, in a room. Read them that way.

Group 1 — Straightforward

One idea, one lesson. Nearly all of these are the same program wearing a different hat — so the difference between them is your design, not your code.

☐ Dice ★

Shake it, get a number from 1 to 6. That is the whole program — and almost everything else on this menu grew out of it.

But a dice on its own is not a game. The real project is to make **two** dice — two micro:bits, or one that rolls twice — and then *teach the class a game that needs them*. Backgammon. Yahtzee. Pig. You are not building a dice; you are running a game where you happen to be the dice.

Upgrade: draw the actual dots instead of a number, like a real dice face. Six patterns, six decisions.

You will need: `on shake · pick random 1 to 6 · show number`. For two dice, a second `pick random and +`. For dots, `six if / else and show leds`.

☐ Coin Flipper ★ ?

Heads or tails, settled forever. One random number, two outcomes, no more arguments.

And now the real question. The micro:bit does not actually own a coin. It runs a formula that *pretends* to be random. So: flip a real coin 100 times and write down the result. Then flip the micro:bit 100 times.

Do you get the same answer?

Careful — 100 real coin flips almost never gives exactly 50. Anything from about 40 to 60 is completely normal. So "not 50" does not prove anything is broken. Work out what *would*.

You will need: `pick random 0 to 1 · if / else · show icon · a tally chart and some patience`

☐ **Magic 8-Ball**

Ask it a question out loud. Shake. It answers. It is never wrong, because it never knew anything in the first place.

Yes, you can write your own answers — and that is the whole project. Swap the picture for `show string` and it can say anything you like. Want twenty answers instead of three? Widen the random range. **The answers are the design.** Make them funny and it is a great game; make them boring and it is a shrug.

You will need: a variable · pick random 1 to 3 (or more) · if / else if / else · show string

☐ **Rock Paper Scissors** ★

Shake your fist and the micro:bit throws for you. Nobody can cheat by waiting half a second any more.

There is a V2 version with sound — a different noise for rock, paper and scissors. Same program, plus the buzzer. Only works on the newer micro:bits, so ask first.

You will need: a variable called `tool` · pick random 0 to 2 · if / else if / else · show leds

☐ **Scorekeeper**

A scoreboard for a game you already play — and the point is the game, not the scoreboard.

Build a **shuffleboard** court down the middle of a table, or skittles, or a paper-ball goal. Then keep score on the micro:bit: B adds a point, A shows the total, both buttons together wipes it back to zero. Whoever brings the best game wins the lesson.

You will need: a variable called `count` · change by 1 · if / else · on button A+B pressed

☐ **Step Counter** ?

Put it in your pocket and it counts every step you take.

Or does it? Walk exactly 100 steps, counting out loud, and see what the micro:bit says. Then try running. Then try going up stairs. Then put it in your bag instead of your pocket.

Find out where it goes wrong, and then *fix it*. That is a far better project than the counting.

You will need: a variable called `steps` · on shake · change by 1 · show number

☐ **Times Tables Tester**

Press A for a number, B for another, then shake to reveal what they multiply to. Fastest way to find out whether you actually know your twelves.

You will need: two variables · pick random 1 to 12 · $\times$ · three different inputs

Group 2 — A step up

These need something that repeats, something that counts time, or a rival sitting opposite you.

☐ Hot Potato

Press A and pass it round the circle as fast as you can. Somewhere between 5 and 15 seconds it dies in somebody's hands, and nobody knows when.

You will need: a variable called timer · pick random 5 to 15 · a while loop counting down · sound or a picture

☐ Sixty Seconds ?

Press A, shut your eyes, and press B when you think a whole minute has gone by. The micro:bit then tells you exactly how wrong you were, in milliseconds.

Almost nobody is any good at this, and that is the interesting bit. Test the whole class. Does everyone guess short, or long? Does it change if you count in your head? Does it change if somebody is talking to you?

You will need: running time · variables start and elapsed · absolute value to ignore whether you were over or under

☐ Reaction Timer ?

Wait for the light. Hit the pad the instant you see it. It shows your reaction time as an actual number of milliseconds — and it catches you if you go early.

Needs building, not just coding: you make touch pads out of kitchen foil and crocodile clips. One hand on the earth pad, the other hand doing the reacting. Add a third pad and two of you can race.

You will need: running time · variables start and end · a **true/false** variable to catch false starts · on pin pressed · foil and crocodile clips

☐ Snap the Dot ★

A dot bounces back and forth across the screen. Press A at the exact moment it hits the middle. Miss, and it is over.

You will need: create sprite · move · if on edge bounce · if sprite x = 2 · add score · game over

☐ **Heads Guess!**

The micro:bit goes on your forehead and shows a word that everybody can see except you. Thirty seconds on the clock. Tilt it forward when you get one, tip it back to pass.

You write the word list, which is where it gets funny. Teachers. Footballers. Things in this room. The list *is* the game — a brilliant list beats a clever program every time.

You will need: a list of words · a variable called `index` · `start` · `countdown` · `tilt` gestures

☐ **Salute!**

Two players hold a micro:bit to their forehead. Each shows a number — and you can see everyone's but your own. A third person calls out the total. First to work out what they must be wins.

Like Heads Guess, but with mental arithmetic instead of words, and it needs three people.

You will need: variables · `on screen up` · `pick random` · `score buttons` · 2–3 micro:bits

☐ **Crashy Bird** ★

Flappy Bird, on a screen five pixels wide. A to go up, B to go down, do not hit the pipes. It should not work, and somehow it does.

You will need: lists to hold the obstacles · sprites · `forever` · collision checks. The hardest one in this group by a distance.

Only on the newer micro:bits — ask before you pick one of these

Some micro:bits in the class have a **microphone and a speaker** built in and some do not. These four need one or the other. Check with Mr. Rawson which board you have before you set your heart on them.

Clap-o-meter — measures how long the applause lasts. Settles, once and for all, who got the biggest cheer. (*microphone*)

Blow Away — blow on it and the lights scatter away from your breath. (*microphone*)

Countdown — a timer that actually beeps at you instead of politely showing a picture. (*speaker*)

Rock Paper Scissors with sound — as above, plus a different noise for each hand. (*speaker*)

Group 3 — Two micro:bits talking

Radio. Your micro:bit sends messages through the air to somebody else's. Ambitious, and you will need a partner.

☐ Two-Way Radio ★ (radio)

A proper handheld radio. Press a button and your message appears on your friend's micro:bit across the room — no wires, no wifi, no phone.

The catch, and it is the whole design problem: there is no keyboard, so every message has to be written into the code *before* you download it. Two buttons, a shake and the logo gives you about four messages. **So which four messages actually matter?** Pick badly and the radio is useless. That is the project.

You will need: `radio set group · radio send string · on radio received · show string`

☐ Morse Chat ★ (radio)

The answer to the problem above. Short press for a dot, long press for a dash — and now you can send *any* message you like, one letter at a time.

Slower. Much more satisfying. You will both have to learn Morse, which is either the best or the worst part depending on who you ask.

You will need: `running time to tell a short press from a long one · a text variable that builds up · radio · a lookup of the alphabet`

☐ Multi Dice (radio)

Everybody shakes at once and every micro:bit finds out who rolled highest. A dice game where the dice argue among themselves.

You will need: `radio · variables dice and received · if`

☐ Tele-potato (radio)

Hot Potato, except the potato flies through the air from micro:bit to micro:bit and you never see it coming.

You will need: `radio · a timer · a loop`

☐ **Assassin (radio) — nobody has built this one**

There is a game called **Group Teleporting Duck** where you shake and a duck flies to a random player somewhere in the room. It works, and it is fine, and it is about a duck.

Here is the better version, which does not exist yet. Everyone gets a micro:bit. One is secretly the *assassin*, one is secretly the *doctor*, everyone else is a villager. The assassin picks somebody; the doctor picks somebody to save; the micro:bits sort out who survives the night. Nobody knows who anybody is.

You would be inventing this, not copying it. That is harder, and it is worth far more.

You will need: radio · a **role** variable · **true/false** variables for alive and dead · player ID numbers · and a set of rules you write yourself

☐ **Hot or Cold (radio)**

One person hides an object and their micro:bit tells the seeker "warmer" or "colder" as they hunt for it.

You will need: radio · **text** messages, not numbers · **two different programs**, one each

☐ **Treasure Hunt (radio)**

Hide beacons around the room. Your micro:bit reads how strong each signal is, so the screen tells you when you are getting close.

You will need: radio **signal strength** · an **id** variable · **two separate programs**

☐ **Red Light Green Light (radio)**

One player is the traffic light and broadcasts to everybody at once. The accelerometer catches you moving on red. There is no arguing with it.

You will need: radio · a **state** variable · the accelerometer · **two programs**

☐ **Infection (radio)**

One player starts infected and nobody knows who. Walk near somebody and you might pass it on. Twenty seconds before symptoms show. Survive to the end.

You will need: radio · **signal strength** · **timers** · **several states**. Needs **five or more** micro:bits. The most ambitious thing on this menu.

Name: _____

Date: _____

Not on the menu, but out there

These exist too. If nothing above grabbed you, go and look — but check with Mr. Rawson before you commit, because some are much bigger jobs than they sound.

Fireflies · Mood Radio · Voting Machine · Best Friends · Micro:Coin · Karel the LED · Pet Hamster · Cat Napping · Clap Lights · Banana Keyboard · Guitar · Stopwatch · Spirit Level · Compass · Nightlight · Jukebox · Sensory Toy · Tilt Alarm · Which Way Now? · Rock Paper Scissors Teams

Where to find them

microbit.org/projects/make-it-code-it

123 projects. You can filter them by how hard they are, and — much more usefully — by **what part of the micro:bit they use**: the buttons, the accelerometer, the radio, the microphone. Set the filters to *Beginner* and *MakeCode* and browse only what you can actually reach.

makecode.microbit.org/projects

Fewer projects, but these are the ones with the full step-by-step tutorials — the ★ ones on this menu. The editor walks you through every single block.

Your choice

First choice _____

Second choice _____

Why the first one? Who would play it?

